

Architecture Documentation

RAG Assistant - End-to-End Pipeline

Overview

The RAG Assistant follows a linear, ten-stage pipeline that takes a user's question and an uploaded knowledge base, and produces a grounded, cited answer. Each stage has a single, well-defined responsibility, which keeps the system modular and easy to debug. The diagram below shows the full flow, followed by a detailed explanation of every component.

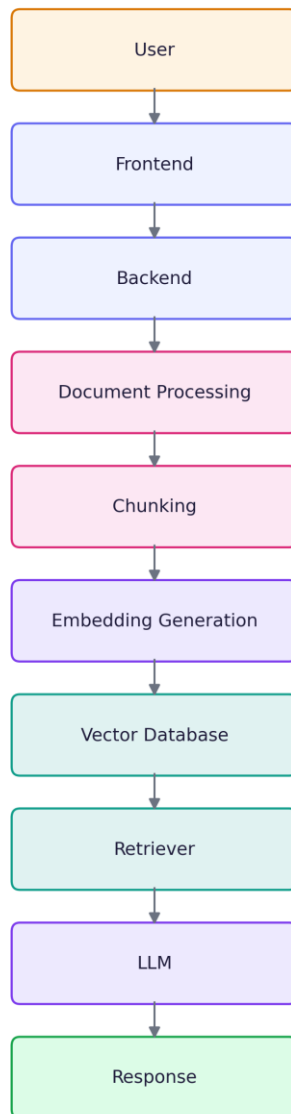
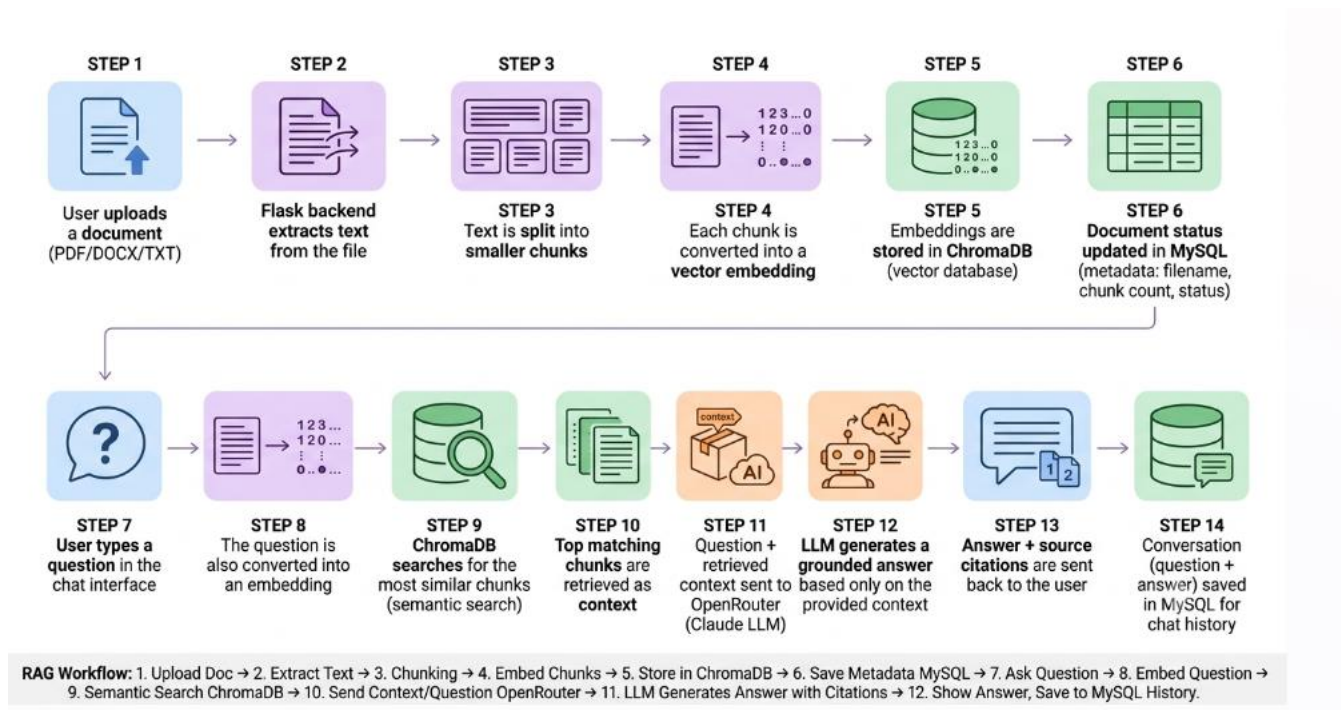


Figure 1: End-to-end system pipeline



1. User

The end user interacts with the system through a web browser. They upload documents to build a knowledge base and type natural-language questions they want answered based on that content.

2. Frontend

Built with HTML, CSS, Bootstrap, and vanilla JavaScript. The frontend renders the chat interface, document upload controls, and admin dashboard. It sends the user's question and uploaded files to the backend via `fetch()` calls, and renders the returned answer, citations, and chat history without reloading the page.

3. Backend

A Flask application that acts as the orchestrator for the entire pipeline. It exposes REST routes for uploading documents, sending chat messages, retrieving history, and viewing admin statistics. The backend coordinates document processing, embedding, retrieval, and LLM calls, and persists structured data (documents, sessions, messages) in MySQL.

4. Document Processing

When a file is uploaded, the backend extracts raw text from it. PDF files are parsed using PyPDF2, and Word documents are parsed using python-docx, plain text files are read directly. The result is a single block of extracted text ready for splitting.

5. Chunking

The extracted text is too large to embed or feed to an LLM as one unit, so it is split into smaller, overlapping chunks (for example, 500 words per chunk with a 50-word overlap). Overlap preserves context across chunk boundaries so that information split across two chunks is not lost during retrieval.

6. Embedding Generation

Each text chunk is converted into a dense numerical vector using a local, free sentence-transformers model (all-MiniLM-L6-v2). This vector captures the semantic meaning of the chunk, allowing similar meanings to be compared mathematically rather than through exact keyword matching.

7. Vector Database

The generated embeddings are stored in ChromaDB, a vector database optimized for similarity search. Each stored vector is linked to its source chunk text and document ID, allowing retrieved matches to be traced back to their original document.

8. Retriever

When a user asks a question, the question itself is embedded using the same model, and the vector database is queried for the top-k most semantically similar chunks. This retrieval step is what grounds the system in real, current knowledge instead of relying only on what the LLM was originally trained on.

9. LLM

The retrieved chunks, along with the user's original question, are assembled into a structured prompt and sent to a large language model (Claude, accessed via the OpenRouter API). The model is explicitly instructed to answer only using the provided context, which reduces the likelihood of fabricated or unsupported answers.

10. Response

The generated answer is returned to the frontend along with the source chunks used to produce it, displayed as citations the user can inspect. The question and answer are also saved to MySQL, allowing the conversation to persist and be revisited later.

Summary

This pipeline demonstrates the core principle of Retrieval-Augmented Generation: rather than asking a language model to answer from memory alone, the system first retrieves relevant, verifiable information and then asks the model to reason over that specific context. Each stage in the pipeline is independently replaceable — for example, the vector database or LLM provider could be swapped without changing the rest of the architecture — which makes the design suitable as a foundation for a production-grade enterprise system.